Final Project: City Emergency Dispatch Simulation

General Description:

The project is aimed at implementing a simulation of a city emergency dispatch system that handles various emergency calls from citizens, such as fires, accidents, or medical incidents. The system includes a central dispatcher that forwards the calls to the relevant departments such as ambulance, police, fire department, and others. Each department manages a limited number of resources to carry out tasks, and it is necessary to prioritize the calls and ensure optimal use of the resources. Additionally, logs should be written to track the actions performed by the system.

Steps to Implement:

1. Event Generation:

- **Description**: Events are generated randomly and simulate emergency calls from citizens. Each event is represented by a message with a specific code (e.g., 1 for police, 2 for the fire department).
    - To generate random events, a timer should be used to trigger events at random intervals (e.g., every 1-5 seconds). Each event will generate a message with a code representing the type of event and insert it into the dispatcher's queue.
    - In a Nucleus board environment, events will be generated from an ISR (Interrupt Service Routine) triggered by a hardware timer. In a simulator, a thread will simulate random emergency events that generate these "interrupts."
    - **Example**: A random event is generated every 1-5 seconds:
        - Event 1 (Code: Police)
        - Event 2 (Code: Ambulance)
        - Event 3 (Code: Fire Department)

2. Dispatcher Queue:

- **Description**: The dispatcher reads messages from the event queue and forwards them to the appropriate departments. The dispatcher operates periodically: at set time intervals (ticks), it reads a message from the queue and forwards it to the relevant department.
    - The dispatcher needs to check the event type (based on its code) and forward it to the appropriate department:
        - Code 1: Police
        - Code 2: Ambulance
        - Code 3: Fire Department

    - You can add a prioritization mechanism so that the dispatcher gives priority to critical events (e.g., fires or medical emergencies). Events will receive a priority score, and the dispatcher will handle the highest-priority events first.
    - **Example**: The dispatcher reads a message from the queue:
        - Event with Code 2 (Ambulance)
        - The dispatcher forwards the call to the ambulance department, which begins to handle the task.

3. Emergency Departments:

- **Description**: Each department manages a limited number of resources and tasks. When the department receives a call from the dispatcher, it comes out of the blocking state, performs the task, and after completion returns to the blocking state to wait for the next call.
    - Each department has a limited number of resources:
        - Ambulance: 4 available ambulances.
        - Police: 3 available police cars.
        - Fire Department: 2 available fire trucks.
        - Corona (Covid-19): 4 resources.
    - Each task takes a random amount of time to complete (ticks). After the task is completed, the resource is released and can be used again.
    - If all the resources of a department are occupied, additional calls will wait in a queue until a resource is available.
    - **Example**: The ambulance department receives a call from the dispatcher. It handles the task for 3 ticks, then the resource is freed and available for the next task.

4. Resource Management:

- **Description**: Managing limited resources efficiently is critical to the success of the simulation. You must ensure that each department does not exceed its allocated resources. If all resources are occupied, calls will remain in the queue until resources are freed.
    - To prevent situations where a department is overwhelmed and cannot handle all calls, a flexible mechanism can be implemented where departments can "borrow" resources from each other in case of heavy load.
    - **Example**: If there aren’t enough available ambulances and other departments are not busy, resources from those departments can be "borrowed" to handle the medical calls.

5. Logging:

- **Description**: Every action in the system should be recorded in a log for tracking. The logs should include details about every call received, which department handled it, how long the task took, and what the outcome was.

    - In an embedded environment like the Nucleus board, `UART Transmit` can be used to send log messages. In a simulator, `printf` can be used to print the logs on the screen.
    - The logs will provide insights into the performance of each department and help analyze the overall system's efficiency.
    - **Example**: "Received call with Code 2 (Ambulance). A free resource was allocated for the task, which lasted 3 ticks. The task was completed."

6. Synchronization and Locking Mechanisms:

- **Description**: Since each department manages limited resources and operates in parallel with the dispatcher and other departments, synchronization and locking mechanisms must be implemented. This ensures that multiple departments do not try to use the same resources simultaneously, preventing conflicts.
    - For example, using mutexes to protect access to resources and prevent competition between threads trying to access the same resource at the same time.
    - **Example**: While the ambulance department is using a resource, no other department can access it until it is released.

7. Fault Management:

- **Description**: You need to handle potential failures in the simulation. For example, if a department fails to complete a task due to resource limitations, a mechanism should be added to continue handling events intelligently.
    - A fault management mechanism can be implemented to try to handle failed tasks, either by redirecting them to other departments or by logging the failure for further processing.
    - **Example**: "The task for the fire department failed due to a lack of resources. The event will remain in the queue for further handling."

Additional Highlights:

- Optimize load management so that the system can function efficiently even when there are large amounts of calls.
- Using a priority queue structure will help respond more quickly to urgent events.
- A simple graphical interface that shows the status of departments and calls could assist in visually analyzing the simulation.

Final Grade Composition:

1. **Code fulfills all requirements (80%)**:
    - The code must implement all the functional requirements of the project. The dispatcher must manage queues, resources, and calls according to the scenarios defined, including handling overloads, dynamic prioritization, and fault management.
2. **Code is neat and organized (10%)**:
    - The code must be clearly written and well-organized. Every function should be well-defined, with meaningful variable names and appropriate comments for documentation.
3. **Job Interview (10%)**:
    - This part will evaluate your ability to explain the decisions you made in the project. You will be asked to present how you managed the queues, resources, and various events, as well as how you handled overload scenarios and faults. Additionally, you will be required to explain how you implemented synchronization and locking mechanisms and provide insights into any improvements made.

Good luck!